

Microservices Tooling & Configuration — Merged (Spring Boot 3.x/“4”-ready)

This document merges: - Modern microservices concepts/tooling from Amazon-Google_Microservices_Interview_Prep_Java-Spring.md (sections 8–9) - Hands-on Spring Boot configuration from Microservices_Tools_Concepts_SpringBoot3.md

Goal: **one production-oriented reference** for tools, concepts, configs, and common annotations.

Version note

The content is written for **Spring Boot 3.x + Spring Security 6 + Spring Cloud current release trains**. If you call it “Spring Boot 4”, treat it as a forward-looking label; the architecture and configs remain the same.

Table of contents

1. Core concepts (modern)
 2. Service discovery (Kubernetes DNS vs Eureka)
 3. API Gateway (Spring Cloud Gateway)
 4. Resilience (Resilience4j)
 5. Observability (OpenTelemetry, metrics, logs)
 6. Messaging/eventing (Kafka + alternatives)
 7. Security baseline (JWT/OIDC)
 8. Deployment & config management
 9. Annotations cheat sheet
 10. Microservices interview question bank (concept-mapped)
-

1) Core concepts (modern)

- Service boundaries and data ownership
- Database-per-service
- Idempotency
- Saga + compensation
- Transactional outbox

- Backpressure + DLQ
 - SLOs/SLIs and error budgets
-

2) Service discovery

2.1 Recommendation

- On Kubernetes (EKS): prefer **Kubernetes Service DNS** (often no Eureka needed).
- Without Kubernetes: consider Eureka or cloud-native discovery.

2.2 Eureka Server (Boot 3) — example

- `@EnableEurekaServer`
- `application.yml` with server port 8761 and server not registering itself

2.3 Eureka Client — example

- `spring-cloud-starter-netflix-eureka-client`
 - `eureka.client.serviceUrl.defaultZone`
 - Optional: `@EnableDiscoveryClient`, `@LoadBalanced RestTemplate`
-

3) API Gateway (Spring Cloud Gateway)

3.1 What it should do

- Routing, authn, rate limiting, request normalization

3.2 Minimal gateway route example

- Configure `spring.cloud.gateway.routes`

3.3 Rate limiting example

- Redis-backed `RequestRateLimiter`
 - `KeyResolver` bean (IP/user/api key)
-

4) Resilience (Resilience4j)

- Concepts: timeouts, bounded retries + jitter, circuit breakers, bulkheads
- Example annotations:
 - `@CircuitBreaker(name=..., fallbackMethod=...)`
 - `@Retry(name=...)`

5) Observability

5.1 Tracing

- OpenTelemetry + OTLP exporter
- Sampling config
- Context propagation across HTTP and Kafka

5.2 Metrics

- Actuator + Micrometer + Prometheus registry
- `management.endpoints.web.exposure.include`

5.3 Logs

- Structured JSON logs
 - correlation IDs + trace IDs
-

6) Messaging/eventing

6.1 Kafka (Spring for Kafka)

- `@KafkaListener`
- manual ack mode example
- poison message strategy (retry topic/DLQ)

6.2 AWS alternatives

- SQS/SNS/EventBridge/Kinesis: when to use each
-

7) Security baseline

- OIDC/JWT validation (gateway and/or services)
 - method-level authorization
 - avoid logging PII
-

8) Deployment & config management

- Docker + K8s manifests

- progressive delivery (canary)
 - Spring Cloud Config vs AWS AppConfig
 - secrets management
-

9) Annotations cheat sheet

- Spring core: @SpringBootApplication, @Configuration, @Bean, @Service
 - Web: @RestController, @GetMapping
 - Discovery: @EnableEurekaServer, @EnableDiscoveryClient, @LoadBalanced
 - Resilience4j: @CircuitBreaker, @Retry, @Bulkhead, @TimeLimiter
 - Kafka: @KafkaListener
-

10) Microservices interview question bank (concept-mapped)

Use the full question bank in your master doc section # 9 as the authoritative list. This merged doc focuses on tooling/config; keep Q&A in the interview doc.

Microservices Tools & Concepts — Spring Boot 3 (and Spring Cloud 2023/2024)

This document is focused on **modern microservices practices** and **how to configure common tooling** with **Spring Boot 3**. For “Spring Boot 4”, Spring has not widely standardized that as a released line at the time of writing; treat “3.x + current Spring Cloud release train” as the practical target.

Table of contents

1. Core concepts (what you must know)
2. Service discovery (Eureka vs Kubernetes vs Cloud)
3. API Gateway (Spring Cloud Gateway)
4. Resilience (Resilience4j)
5. Distributed tracing (OpenTelemetry)
6. Metrics and monitoring (Micrometer/Prometheus)
7. Centralized logging
8. Security (OAuth2 resource server, mTLS notes)

9. Messaging/eventing (Kafka)
 10. Configuration management
 11. Spring Boot annotations cheat sheet (what/when)
-

1) Core concepts (what you must know)

1.1 Key concepts

- **Service boundaries (DDD):** each service owns its data and lifecycle.
- **Database-per-service:** avoid shared database coupling.
- **Async-first where appropriate:** queues/streams decouple services.
- **Idempotency:** required for safe retries.
- **Saga + compensation:** distributed workflow without 2PC.
- **Outbox pattern:** publish events reliably with DB writes.
- **Observability:** logs + metrics + traces; define SLOs.

1.2 Modern tool categories

- **Discovery:** Kubernetes DNS, Cloud Map, Eureka (legacy/common in Spring environments)
 - **Gateway:** Spring Cloud Gateway, Envoy/Ingress, API Gateway (managed)
 - **Resilience:** Resilience4j, service mesh policy
 - **Tracing:** OpenTelemetry, Tempo/Jaeger, AWS X-Ray
 - **Metrics:** Micrometer, Prometheus, Grafana
 - **Logs:** JSON logs, OpenSearch/ELK/Loki
-

2) Service Discovery

2.1 What service discovery solves

- Decouples clients from fixed host/port lists.
- Enables scaling, rolling deploys, and failover.

2.2 Modern recommendation (2026-ready)

- **If you are on Kubernetes (EKS/GKE/AKS):** prefer **Kubernetes Services + DNS** for discovery.
- **If you are not on Kubernetes:** use cloud-native discovery (e.g., **AWS Cloud Map**) or a managed gateway/load balancer.

- **Eureka** is still asked in interviews and used in many Spring Cloud stacks, but in Kubernetes it's often unnecessary.

2.3 Eureka Server (Spring Boot 3) — minimal setup

Gradle/Maven dependency (Eureka Server)

Use Spring Cloud dependency management.

pom.xml (important parts)

- Use a Spring Cloud BOM matching your Spring Boot version.
- Add Eureka server starter.

Example (structure): - spring-cloud-dependencies (BOM) - spring-cloud-starter-netflix-eureka-server

Example pom.xml snippet (Boot 3 + Spring Cloud)

Use this pattern (adjust versions to your project standard):

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>2023.0.3</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
</dependencies>
```

Application code

Create a Spring Boot application and enable Eureka server:

- Add `@EnableEurekaServer` to your main class.

Example

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
```

```
@SpringBootApplication
@EnableEurekaServer
public class DiscoveryServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(DiscoveryServerApplication.class, args);
    }
}
```

Configuration (application.yml)

Typical configuration: - run server on 8761 - disable self-registration for the server

Example keys to know: - server.port: 8761 - eureka.client.register-with-eureka: false - eureka.client.fetch-registry: false

Example

```
server:
  port: 8761

spring:
  application:
    name: discovery-server

eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
```

Operational notes

- Run at least 2 instances (HA) behind a load balancer in production.
- Prefer short TTLs and health checks.

2.4 Eureka Client (Spring Boot 3) — minimal setup

Dependency

Add: - spring-cloud-starter-netflix-eureka-client

Example pom.xml snippet

```
<dependencies>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
```

```

</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
</dependencies>

```

Configuration (application.yml)

Key settings: - spring.application.name - eureka.client.serviceUrl.defaultZone to point to server

Example

```

server:
  port: 8081

spring:
  application:
    name: order-service

eureka:
  client:
    serviceUrl:
      defaultZone: http://localhost:8761/eureka/

```

Common annotations you'll use

- @EnableDiscoveryClient (often optional; auto-config usually picks it up)
- @LoadBalanced on a RestTemplate bean (if using client-side discovery load balancing)

Example:

```

import org.springframework.cloud.client.loadbalancer.LoadBalanced;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestTemplate;

@Configuration
class HttpClients {
    @Bean
    @LoadBalanced
    RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

```


Notes

- Discovery works well for **internal** service-to-service communication.
 - If you're in Kubernetes, evaluate replacing Eureka with DNS.
-

3) API Gateway (Spring Cloud Gateway)

3.1 What an API gateway should do

- Routing to backend services
- AuthN/AuthZ enforcement (often)
- Rate limiting
- Request/response normalization

Avoid: - heavy business logic - tight coupling to internal data models

3.2 Spring Cloud Gateway — minimal setup

Dependency

Add: - spring-cloud-starter-gateway

Example pom.xml snippet

```
<dependencies>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
</dependencies>
```

Routes (application.yml)

Common route model: - spring.cloud.gateway.routes[0].id -
spring.cloud.gateway.routes[0].uri - spring.cloud.gateway.routes[0].predicates
- spring.cloud.gateway.routes[0].filters

Typical examples to know: - Path predicate: Path=/orders/** - StripPrefix filter -
RequestRateLimiter filter (usually Redis-backed)

Example routes

```
server:
  port: 8080
```

```

spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: http://localhost:8081
          predicates:
            - Path=/orders/**
          filters:
            - StripPrefix=0

```

Rate limiting

Common approach: - Use Redis (local or managed) for token bucket. - Configure a KeyResolver (by IP, user, API key).

Example: Redis rate limiting (Gateway)

Dependencies: - spring-boot-starter-data-redis-reactive

Example config:

```

spring:
  data:
    redis:
      host: localhost
      port: 6379
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: http://localhost:8081
          predicates:
            - Path=/orders/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10
                redis-rate-limiter.burstCapacity: 20
                key-resolver: "#{@ipKeyResolver}"

```

Example KeyResolver bean:

```

import org.springframework.cloud.gateway.filter.ratelimit.KeyResolver;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import reactor.core.publisher.Mono;

```

```

@Configuration
class RateLimitConfig {
    @Bean
    KeyResolver ipKeyResolver() {
        return exchange -> Mono.just(
            exchange.getRequest().getRemoteAddress() != null
                ? exchange.getRequest().getRemoteAddress().getAddress().getHostAd
dress()
                : "unknown");
    }
}

```

Common annotations you'll use

- @SpringBootApplication
- @Configuration
- @Bean

Production notes

- Put WAF/CloudFront in front for coarse protection.
 - Ensure timeouts and max body sizes are set.
-

4) Resilience (Resilience4j)

4.1 Concepts interviewers expect

- Timeouts must exist at every hop.
- Retries should be bounded and jittered.
- Circuit breakers prevent cascading failure.
- Bulkheads isolate failures.

4.2 Resilience4j with Spring Boot 3

Dependencies

Add: - resilience4j-spring-boot3

Example pom.xml snippet

```

<dependencies>
    <dependency>
        <groupId>io.github.resilience4j</groupId>
        <artifactId>resilience4j-spring-boot3</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-aop</artifactId>
    </dependency>
</dependencies>

```

```
</dependency>
</dependencies>
```

Configuration (application.yml)

You typically configure: - resilience4j.circuitbreaker.instances.<name> -
resilience4j.retry.instances.<name> - resilience4j.timelimiter.instances.<name>

Example configuration + annotations

```
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        slidingWindowSize: 20
        minimumNumberOfCalls: 10
        failureRateThreshold: 50
        waitDurationInOpenState: 10s
  retry:
    instances:
      paymentService:
        maxAttempts: 3
        waitDuration: 200ms
```

Example usage:

```
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
import io.github.resilience4j.retry.annotation.Retry;
import org.springframework.stereotype.Service;
```

```
@Service
class PaymentClient {

    @CircuitBreaker(name = "paymentService", fallbackMethod = "fallback")
    @Retry(name = "paymentService")
    public String charge(String orderId) {
        // call downstream
        return "OK";
    }

    private String fallback(String orderId, Throwable ex) {
        return "PENDING";
    }
}
```

Common annotations you'll use

- @CircuitBreaker, @Retry, @Bulkhead, @TimeLimiter
- @Service, @Component

Notes

- Don't combine aggressive retries with low timeouts.
 - Monitor open/half-open breaker events.
-

5) Distributed Tracing (OpenTelemetry)

5.1 Modern approach

- Prefer **OpenTelemetry** for vendor-neutral instrumentation.
- Export traces to Jaeger/Tempo, or to cloud-native backends.

5.2 Spring Boot 3 tracing setup (typical)

Dependencies

Common approaches: - Use Micrometer tracing bridge + OTLP exporter.

Example dependencies (Micrometer tracing + OTLP)

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
  <dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-otel</artifactId>
  </dependency>
  <dependency>
    <groupId>io.opentelemetry</groupId>
    <artifactId>opentelemetry-exporter-otlp</artifactId>
  </dependency>
</dependencies>
```

Configuration (application.yml)

Key concepts: - enable tracing - configure OTLP endpoint (collector) - propagate W3C trace context

Example configuration

```
management:
  tracing:
    sampling:
      probability: 1.0

otlp:
```

```
tracing:  
  endpoint: http://localhost:4318/v1/traces
```

Example: adding trace IDs to logs

If you use Logback, ensure your log pattern includes trace/span IDs (exact keys depend on your logging setup).

Async propagation

- Ensure trace context is propagated across:
 - HTTP client calls
 - Kafka/SQS message headers
-

6) Metrics and Monitoring (Micrometer/Prometheus)

6.1 What to measure

- RED metrics per endpoint
- JVM metrics (heap, GC, threads)
- connection pools
- dependency latency

6.2 Spring Boot 3 setup

Dependencies

- spring-boot-starter-actuator
- micrometer-registry-prometheus (if using Prometheus)

Example pom.xml

```
<dependencies>  
  <dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-actuator</artifactId>  
  </dependency>  
  <dependency>  
    <groupId>io.micrometer</groupId>  
    <artifactId>micrometer-registry-prometheus</artifactId>  
  </dependency>  
</dependencies>
```

Configuration

- expose actuator endpoints
- secure actuator endpoints

Example application.yml

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      probes:
        enabled: true
```

Common annotations you'll use

- @Timed (Micrometer) on methods/endpoints (optional)
 - @RestController and @GetMapping for health-like custom endpoints (if needed)
-

7) Centralized Logging

7.1 Recommendations

- Use structured JSON logs.
- Include correlation/trace IDs.
- Avoid PII.

7.2 Backends

- OpenSearch/ELK
 - Loki
 - CloudWatch Logs
-

8) Security

8.1 OAuth2 Resource Server (common)

- Validate JWTs at the gateway or at services (or both).
- Enforce authorization consistently.

8.2 mTLS (service-to-service)

- Often implemented via service mesh (Istio/Linkerd/App Mesh).
-

9) Messaging/Eventing (Kafka)

9.1 Core concepts

- partitions, consumer groups
- at-least-once delivery
- idempotent consumers
- DLQ strategy (or retry topics)

9.2 Spring for Kafka (Spring Boot 3)

- configure consumer concurrency to partitions
- handle poison messages
- ensure proper commit strategy

9.3 Kafka configuration + example

Dependencies

```
<dependencies>
  <dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId>
  </dependency>
</dependencies>
```

Example application.yml

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: order-consumer
      auto-offset-reset: earliest
      enable-auto-commit: false
    listener:
      ack-mode: manual
```

Example consumer

```
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.support.Acknowledgment;
import org.springframework.stereotype.Component;
```

```
@Component
```

```
class OrderEventsConsumer {
```

```
    @KafkaListener(topics = "order-events", groupId = "order-consumer")
    public void onMessage(String payload, Acknowledgment ack) {
```



```
// 1) validate
// 2) process
// 3) ack only after successful processing
ack.acknowledge();
}
}
```

Poison message handling (what to explain in interviews)

- Use retries with backoff (retry topic) or a DLQ.
- Persist “failed events” with correlation IDs.
- Never block an entire partition indefinitely.

Common annotations you'll use

- `@KafkaListener`
 - `@Component`, `@Service`
-

10) Configuration management

10.1 Patterns

- externalized configuration
- dynamic config with safe defaults
- feature flags

10.2 Tools

- Spring Cloud Config (common)
 - AWS AppConfig (common)
 - LaunchDarkly/Unleash
-

11) Spring Boot annotations cheat sheet (what/when)

11.1 Spring core

- `@SpringBootApplication`
 - Use on the main application class.
- `@Configuration`
 - Defines configuration class that declares beans.
- `@Bean`
 - Declares a bean method in a config class.
- `@Component`, `@Service`, `@Repository`

- Component scanning stereotypes.
- `@Value`, `@ConfigurationProperties`
 - Bind configuration values.

11.2 Web/API

- `@RestController`
 - REST endpoints.
- `@RequestMapping`, `@GetMapping`, `@PostMapping`
 - Map HTTP routes.
- `@Validated`, `@Valid`
 - Request validation.

11.3 Discovery

- `@EnableEurekaServer`
 - Turns a Spring Boot app into a Eureka server.
- `@EnableDiscoveryClient`
 - Enables discovery client behavior (often optional now).
- `@LoadBalanced`
 - Enables client-side load balancing for `RestTemplate`.

11.4 Resilience4j

- `@CircuitBreaker`
- `@Retry`
- `@Bulkhead`
- `@TimeLimiter`

Use these on service methods that call downstream dependencies.

11.5 Messaging

- `@KafkaListener`
 - Consumes Kafka messages.

11.6 Scheduling and async (common patterns)

- `@Scheduled`
 - Cron/fixed-rate jobs.
- `@Async`
 - Async execution (ensure proper executor configuration).